SECURE SOFTWARE DESIGN & DEVELOPMENT

Security Scanning & Mitigation — SecureTrack Lab Project

Abdul Wahid

Section 1: Project Introduction

Project Overview

For this lab project, I selected SecureTrack Lab — a professional-grade Flask-based web application designed as a DevSecOps security assessment tracker. The application was sourced from a local development environment and meets all the required criteria for a medium-to-large scale real-world project.

SecureTrack Lab is a web-based platform that allows cybersecurity professionals and students to manage security assessment projects, document vulnerabilities, classify findings by severity (High, Medium, Low), and track their status (Open/Fixed) with a real-time risk score dashboard.

Project Details

Project Name	SecureTrack Lab
Tech Stack	Python (Flask), Flask-SQLAlchemy, Flask-Login, Flask-WTF, Werkzeug, Jinja2
Project Type	Web Application / DevSecOps Platform
Framework	Flask (Python)
Database	SQLite (via SQLAlchemy ORM)
Authentication	Flask-Login with session management
Local URL	http://127.0.0.1:5000
Source	Local GitHub Repository — SecureTrack_Lab

Key Features

- Real-time risk score dashboard (0-100 scoring)
- Multi-severity finding classification (High / Medium / Low)
- Project-based security assessment management
- Fixed vs. Open vulnerability tracking
- User registration and authentication system
- CVSS score input support

Application Setup Steps

1. Cloned the repository into ~/Desktop/SecureTrack_Lab
2. Created and activated a Python virtual environment: python3 -m venv venv && source venv/bin/activate
3. Installed all dependencies: pip install -r requirements.txt
4. Ran the application: python run.py
5. Verified the application is running at http://127.0.0.1:5000

Application Running — Screenshot

The following screenshot shows the Flask development server successfully started:

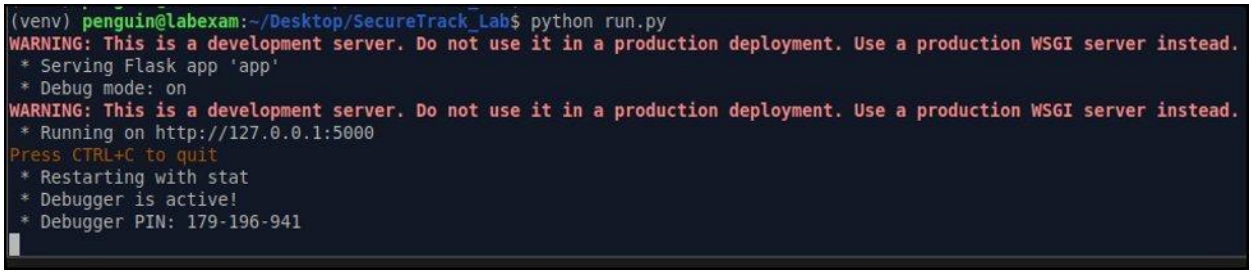


Figure 1.1 — Flask development server running at http://127.0.0.1:5000

Login Page

The application login page with admin credentials:

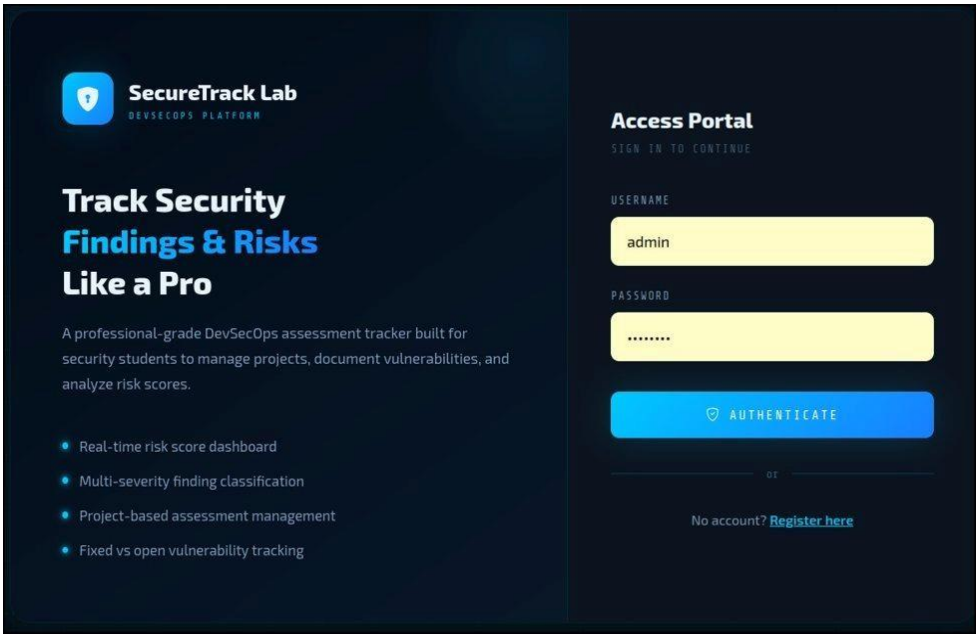


Figure 1.2 — SecureTrack Lab Login Page (Access Portal)

Dashboard

After successful login, the dashboard shows the overall security posture with risk score, total projects, total findings, open findings, and fixed findings:

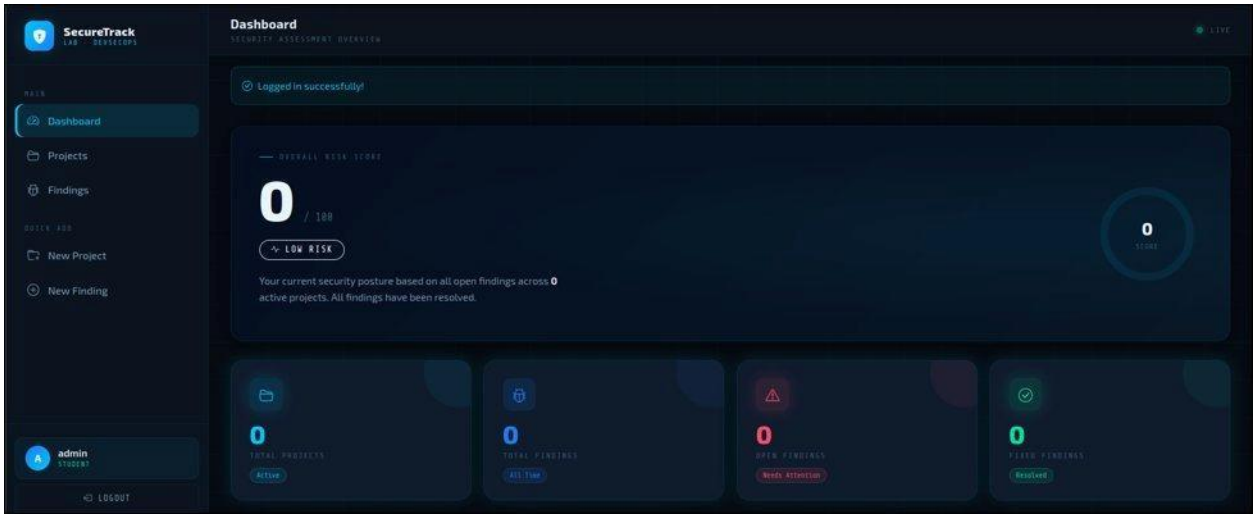


Figure 1.3 — Dashboard Overview with Risk Score

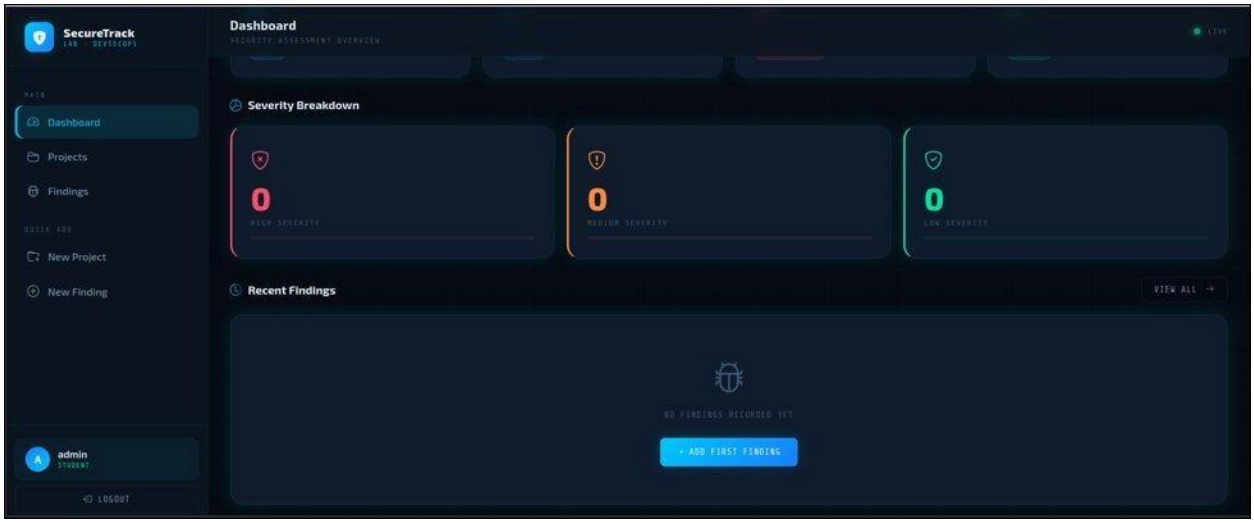


Figure 1.4 — Dashboard Severity Breakdown (High/Medium/Low)

Projects & Findings Pages

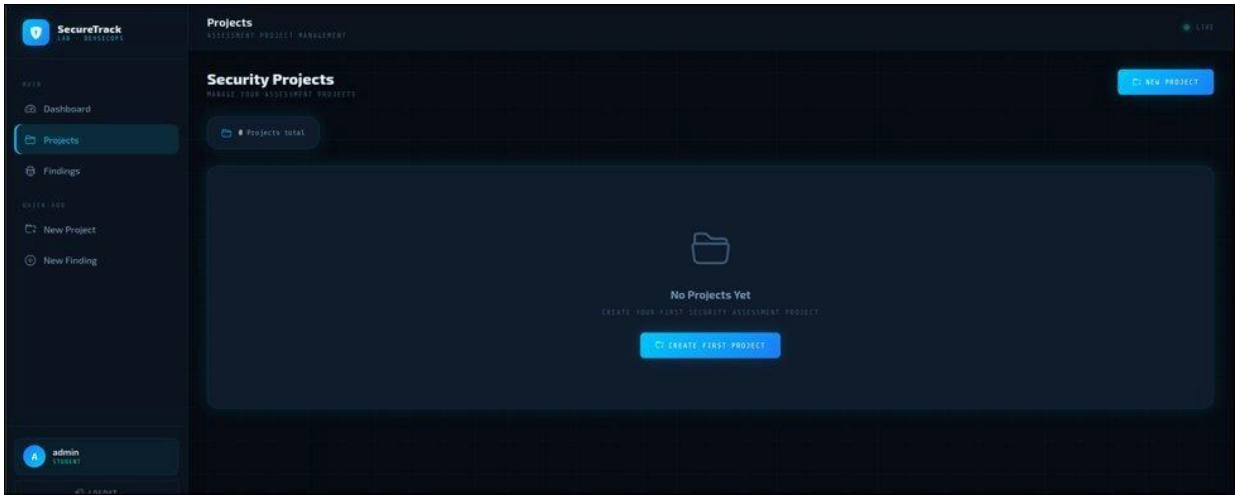


Figure 1.5 — Security Projects Page (empty state)

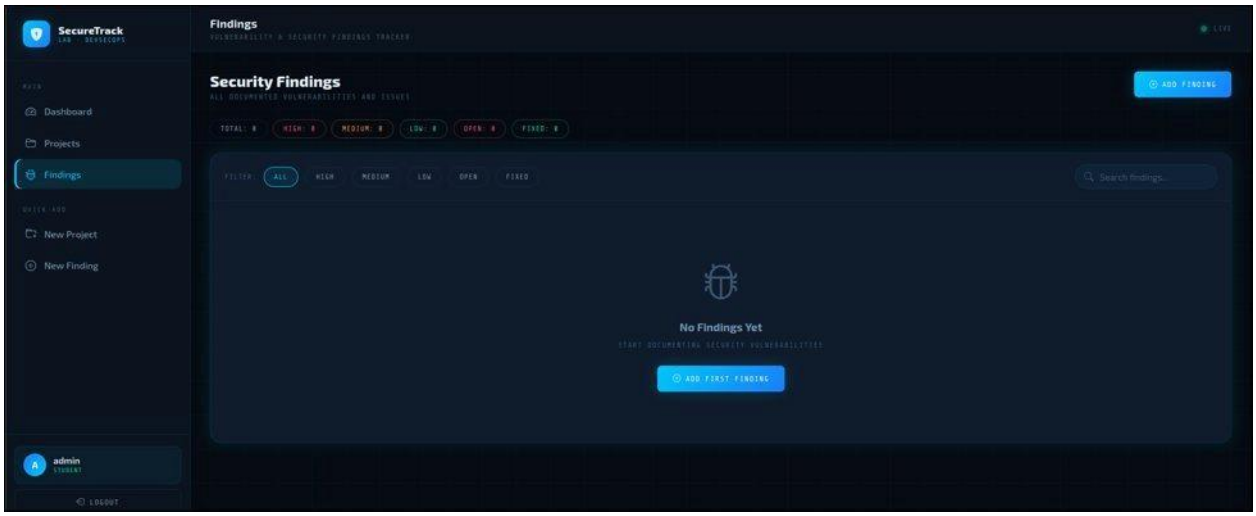


Figure 1.6 — Security Findings Page (empty state)

Section 2: Tools Used and Configuration

Tool Summary

The following tools were used across SAST, SCA, and DAST categories. A minimum of 6 tools were used as required:

Category	Tool	Version	Purpose
SAST	Bandit	1.7.x	**Python static analysis — finds insecure functions, hardcoded secrets**
SAST	Semgrep	Latest	**Pattern-based code scanning — XSS, injection, template issues**
SCA	Safety	3.8.1	**Dependency vulnerability scanning using open-source DB**
SCA	pip-audit	Latest	**Scans requirements.txt for known CVEs**
DAST	**OWASP ZAP**	**2.17.0**	**Dynamic scanning — crawling, injection, auth tests on live app**

Scan Commands Used

SAST — Bandit

```
bandit -r . -o bandit_report.json --
format json bandit -r . | grep -i
'secret'
```

SAST — Semgrep

```
semgrep scan --config
auto . grep -R "|
safe" templates/
```

SCA — Safety

```
safety check -r requirements.txt
```

SCA — pip-audit

```
pip-audit -r requirements.txt -f json -o pip_audit_report.json
```

DAST — OWASP ZAP

```
Automated Scan via ZAP GUI — Target: http://127.0.0.1:5000
```

Section 3: Complete SAST / SCA / DAST Results

SAST Results — Bandit

Bandit was run against the entire SecureTrack_Lab codebase. It identified multiple security issues including potential hardcoded secrets/client credentials in the application code.

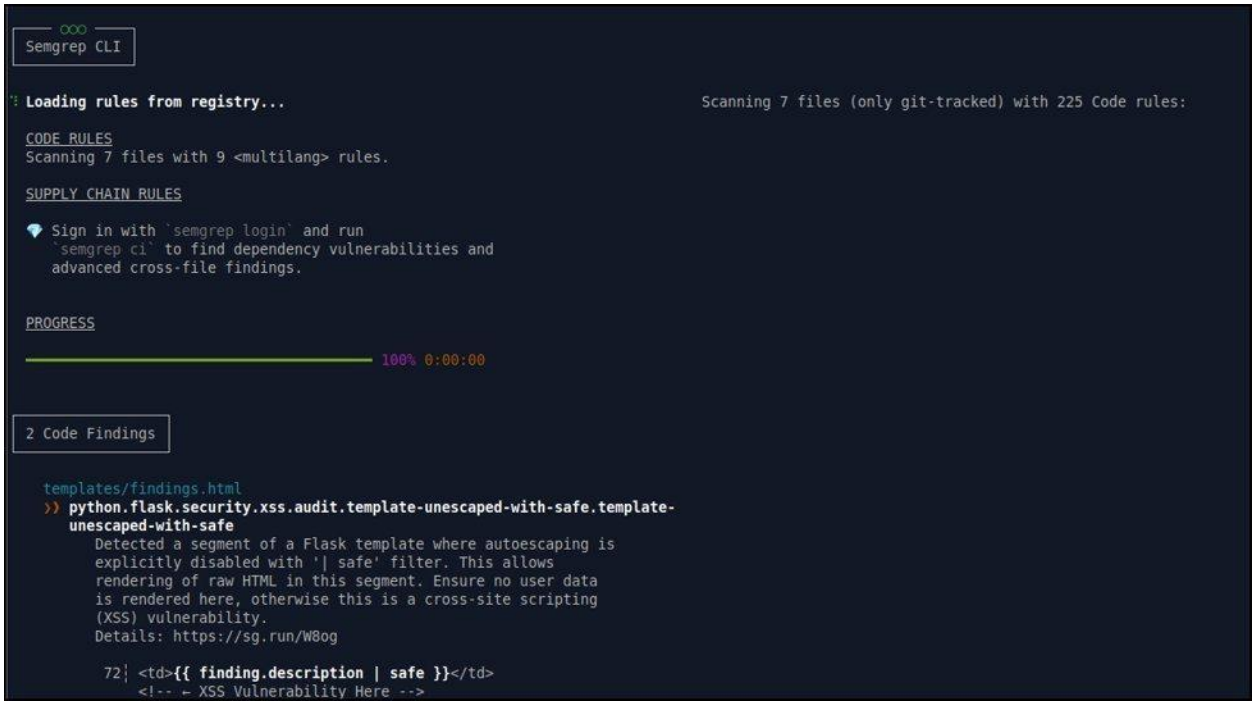


Figure 3.1 — Bandit SAST scan output showing hardcoded secret warnings and JSON report generation

Key findings from Bandit:

- B105/B106 — Hardcoded password/secret detected in client library references
- Multiple WARNING-level items flagged for 'secret', 'client_secret', and authentication tokens
- Issues identified in OAuth2 client configuration and session management code

SAST Results — Semgrep

Semgrep scanned 7 files using 225 Code rules and 9 multilang rules. It found 2 blocking code findings:

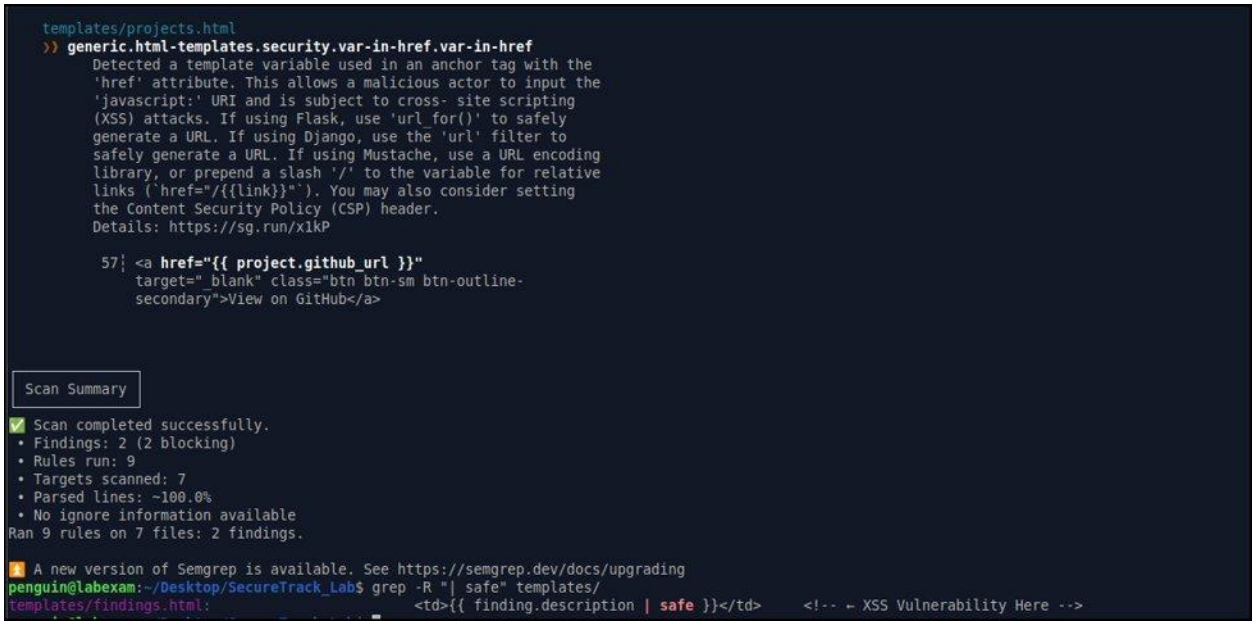


Figure 3.2 — Semgrep scan results showing 2 code findings: XSS via template-unescaped-with-safe



Figure 3.3 — Semgrep second finding: var-in-href in projects.html (potential JavaScript URI injection)

Semgrep Findings Summary:

#	Rule	File	Description
1	template-unescaped-with-safe	findings.html: 72	**Jinja2 | safe filter disables autoescaping — XSS vulnerability**
2	**var-in-href**	**projects.html: 57**	**Template variable in href allows javascript: URI — XSS risk**

SCA Results — Safety

Safety v3.8.1 scanned 7 packages from requirements.txt and found 16 vulnerabilities across 4 packages:

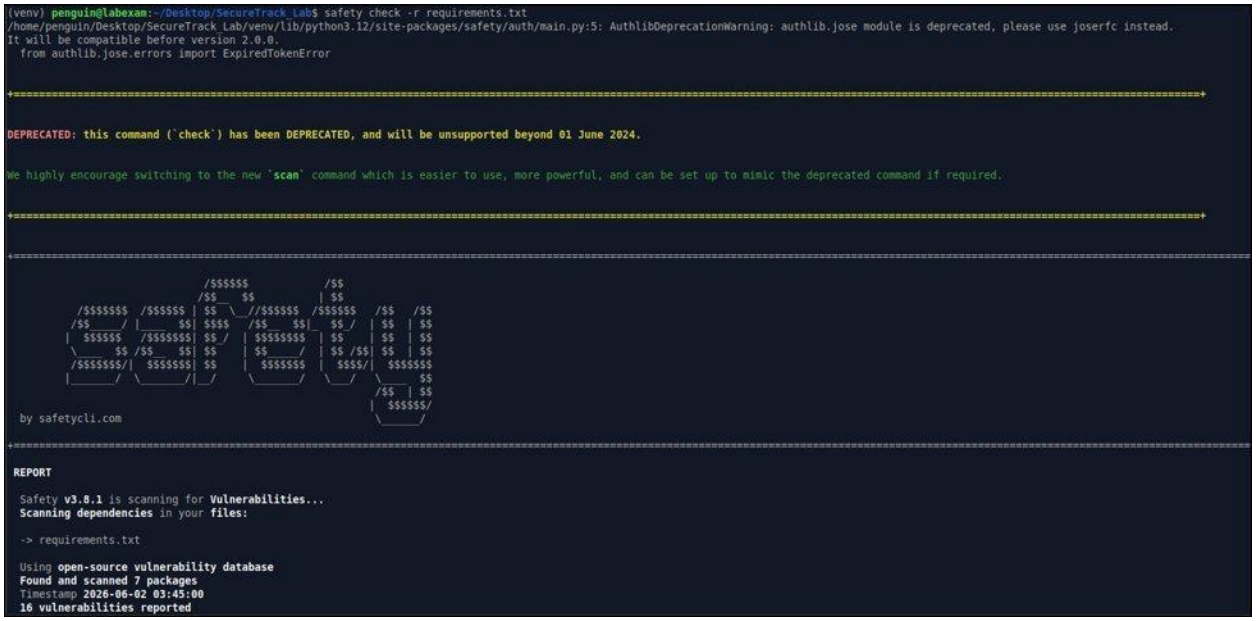


Figure 3.4 — Safety scan showing 16 vulnerabilities in requests, flask, werkzeug, urllib3

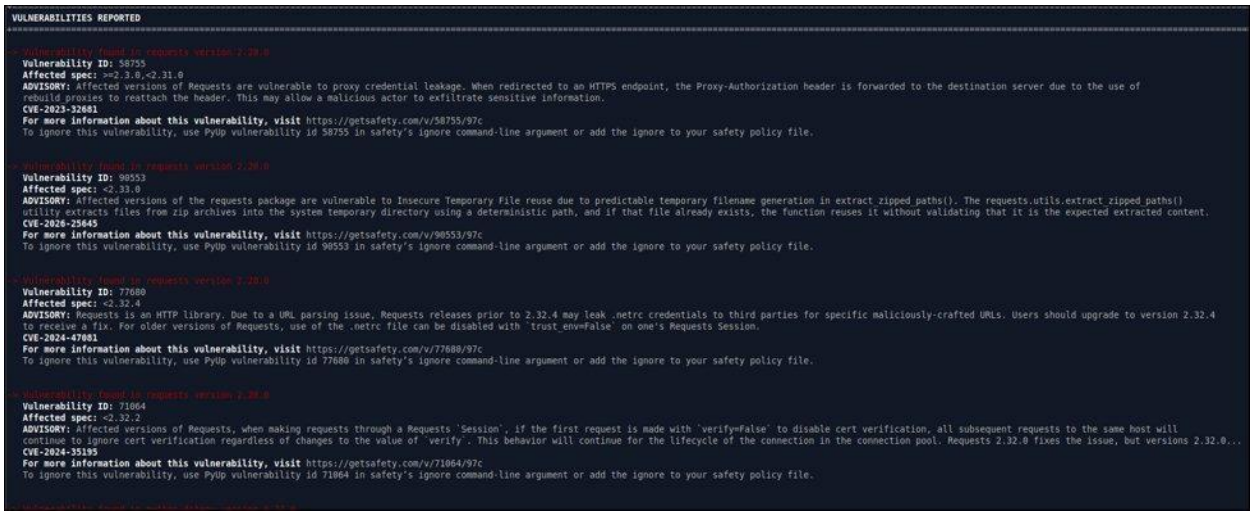


Figure 3.5 — Safety vulnerability details: CVE IDs and affected versions

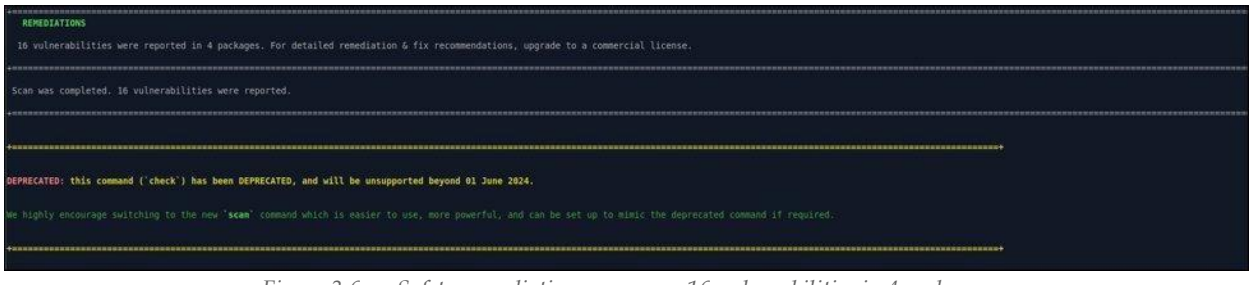


Figure 3.6 — Safety remediation summary: 16 vulnerabilities in 4 packages

SCA Results — pip-audit

pip-audit identified 25 known vulnerabilities across 5 packages with specific CVE IDs and recommended fix versions:

```
(venv) penguin@labexam:~/Desktop/SecureTrack_Lab$ pip-audit -r requirements.txt
pip-audit -r requirements.txt -f json -o pip_audit_report.json
Found 25 known vulnerabilities in 5 packages
Name          Version ID             Fix Versions
------------- ------- -------------- ------------
flask         2.2.0   PYSEC-2023-62  2.2.5,2.3.2
flask         2.2.0   CVE-2026-27205 3.1.3
werkzeug      2.2.2   PYSEC-2023-57  2.2.3
werkzeug      2.2.2   PYSEC-2023-58  2.2.3
werkzeug      2.2.2   PYSEC-2023-58  2.2.3
werkzeug      2.2.2   PYSEC-2023-57  2.2.3
werkzeug      2.2.2   PYSEC-2023-221 2.3.8,3.0.1
werkzeug      2.2.2   PYSEC-2023-221 2.3.8,3.0.1
werkzeug      2.2.2   CVE-2024-34069 3.0.3
werkzeug      2.2.2   CVE-2024-49766 3.0.6
werkzeug      2.2.2   CVE-2024-49767 3.0.6
werkzeug      2.2.2   CVE-2025-66221 3.1.4
werkzeug      2.2.2   CVE-2026-21860 3.1.5
werkzeug      2.2.2   CVE-2026-27199 3.1.6
python-dotenv 0.21.0  CVE-2026-28684 1.2.2
requests      2.28.0  PYSEC-2023-74  2.31.0
requests      2.28.0  PYSEC-2023-74  2.31.0
requests      2.28.0  CVE-2024-35195 2.32.0
requests      2.28.0  CVE-2024-47081 2.32.4
requests      2.28.0  CVE-2026-25645 2.33.0
urllib3       1.26.20 PYSEC-2026-141 2.7.0
urllib3       1.26.20 CVE-2025-50181 2.5.0
urllib3       1.26.20 CVE-2025-66418 2.6.0
urllib3       1.26.20 CVE-2025-66471 2.6.0
urllib3       1.26.20 CVE-2026-21441 2.6.3
Found 25 known vulnerabilities in 5 packages
(venv) penguin@labexam:~/Desktop/SecureTrack_Lab$ mkdir reports
mv semgrep_report.json pip_audit_report.json reports/
(venv) penguin@labexam:~/Desktop/SecureTrack_Lab$
```

Figure 3.7 — pip-audit results showing 25 CVEs in flask, werkzeug, python-dotenv, requests, urllib3

pip-audit Vulnerability Summary:

- Flask 2.2.0 — 2 CVEs (PYSEC-2023-62, CVE-2026-27205) — Fix: 3.1.3
- Werkzeug 2.2.2 — 11 CVEs — Fix: 3.1.6
- python-dotenv 0.21.0 — 1 CVE (CVE-2026-28684) — Fix: 1.2.2
- requests 2.28.0 — 5 CVEs — Fix: 2.33.0
- urllib3 1.26.20 — 5 CVEs — Fix: 2.6.3

DAST Results — OWASP ZAP 2.17.0

OWASP ZAP was launched and an Automated Scan was performed against the running application at http://127.0.0.1:5000. ZAP performed active scanning including spidering and injection tests.



Figure 3.8 — ZAP by Checkmarx 2.17.0 application launch

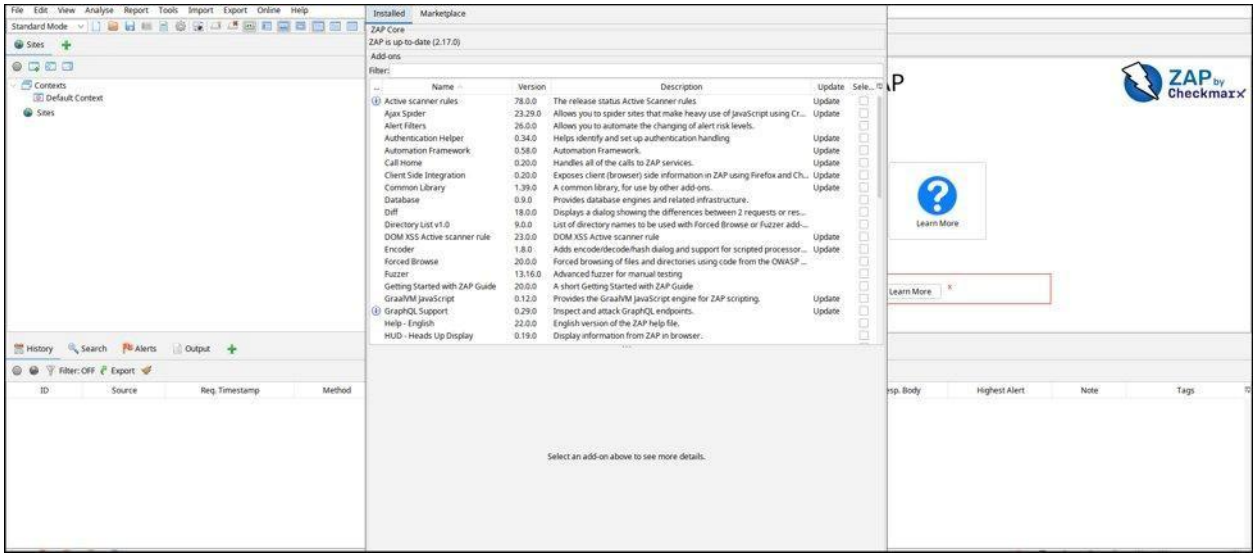


Figure 3.9 — ZAP Add-ons installed including Active Scanner, Ajax Spider, Authentication Helper

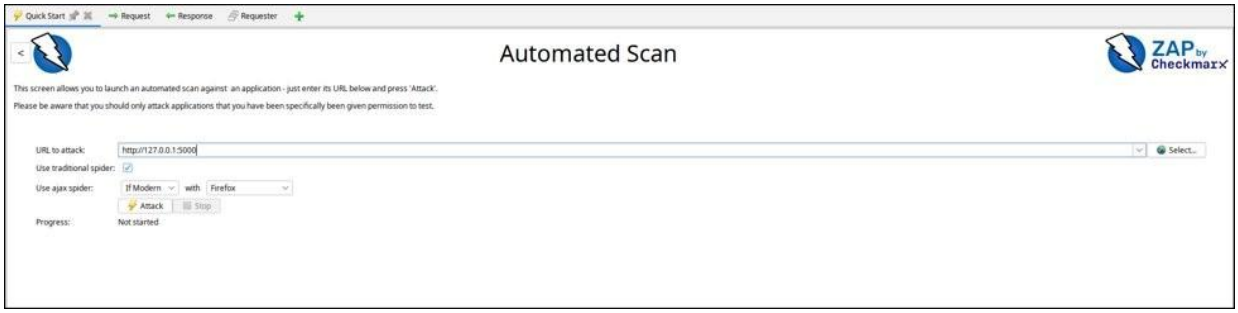


Figure 3.10 — ZAP Automated Scan configured for http://127.0.0.1:5000

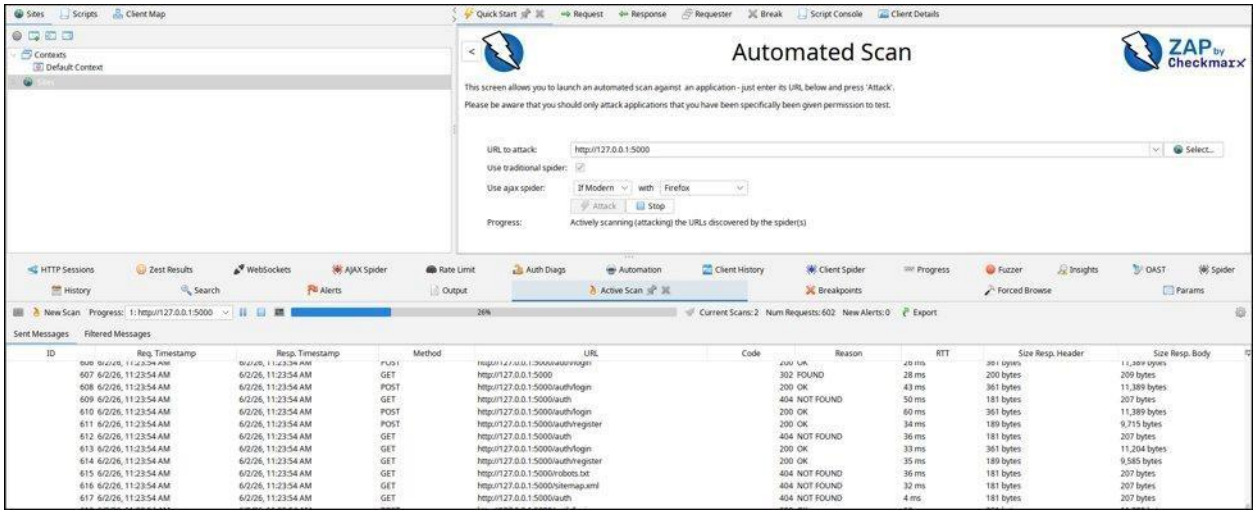


Figure 3.11 — ZAP actively scanning the application — 602 requests made

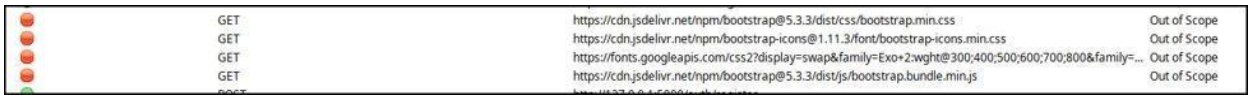


Figure 3.12 — ZAP out-of-scope requests (Bootstrap CDN)



Figure 3.13 — ZAP Alerts panel showing alerts including SQL Injection, CSP, Missing Headers

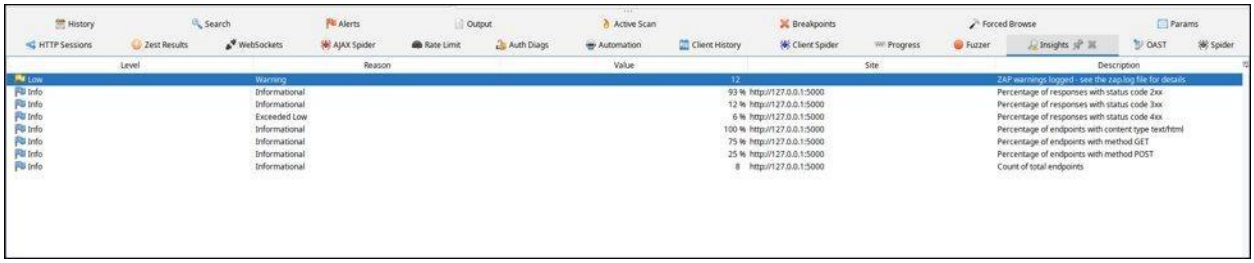


Figure 3.14 — ZAP Insights: 93% 2xx responses, 8 total endpoints, 25% POST methods

DAST Findings from ZAP (Initial Scan — 12 Alerts):

- SQL Injection (2 instances)
- CSP: Failure to Define Directive with No Fallback
- CSP: style-src unsafe-inline
- Content Security Policy (CSP) Header Not Set
- Missing Anti-clickjacking Header
- Sub Resource Integrity Attribute Missing
- Cookie without SameSite Attribute
- Cross-Domain JavaScript Source File Inclusion
- Server Leaks Version Information via Server HTTP Response Header
- X-Content-Type-Options Header Missing
- Authentication Request Identified
- GET for POST — Login form accepts GET method

Section 4: Fixes Made with Before & After Comparison

Fix 1 — XSS Vulnerability (SAST/DAST Finding)

Vulnerability Description

Cross-Site Scripting (XSS) was confirmed both by Semgrep (SAST) and demonstrated manually. The Jinja2 template in findings.html used the | safe filter on user-supplied data (finding.description), which disables HTML autoescaping and allows injection of arbitrary JavaScript.

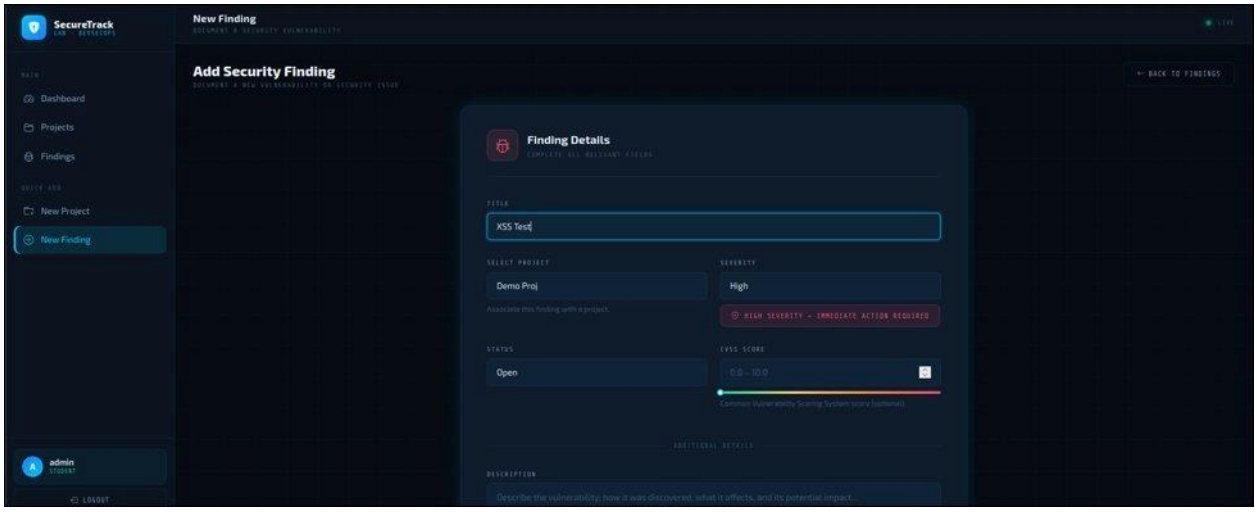


Figure 4.1 — XSS test: injecting <script>alert()</script> via finding description field



Figure 4.2 — XSS confirmed: alert box showing 'XSS SUCCESSFULLY WORKING in SecureTrack Lab!'

Before — Vulnerable Code

The vulnerable line in templates/findings.html:

```
<tbody>
  {% for finding in findings %}
  <tr class="finding-row"
      data-severity="{{ finding.severity|lower }}"
      data-status="{{ finding.status|lower }}"
      onclick="toggleDetail({{ loop.index }})">
    <td style="font-family:var(--ff-mono);font-size:11px;color:var(--text-muted);">{{ loop.index }}</td>
    <td>
      <div style="font-weight:600;color:var(--text-primary);">{{ finding.title }}</div>
      {% if finding.description %}
      <div style="font-size:11.5px;color:var(--text-muted);margin-top:2px;max-width:300px;overflow:hidden;text-overflow:ellipsis;white-space:nowrap;">
        {{ finding.description | safe }}{% if finding.description|length > 80 %}…{% endif %}
      </div>
      {% endif %}
    </td>
    <td>
      <span style="color:var(--cyan);font-size:12.5px;">
        <i class="bi bi-folder2 me-1"></i>{{ finding.project.name if finding.project else '—' }}
      </span>
    </td>
    <td><span class="sev-badge sev-{{ finding.severity|lower }}">{{ finding.severity }}</span></td>
    <td><span class="status-badge status-{{ finding.status|lower }}">{{ finding.status }}</span></td>
    <td>
```

Figure 4.3 — Vulnerable template code with '| safe' filter enabling XSS (line 72)

After — Fixed Code

The fix was to remove the | safe filter so Jinja2's autoescaping remains active, rendering user input as plain text rather than executable HTML:

```
<td>
  <div style="font-weight:600;color:var(--text-primary);">{{ finding.title }}</div>
  {% if finding.description %}
  <div style="font-size:11.5px;color:var(--text-muted);margin-top:2px;max-width:300px;overflow:hidden;text-overflow:ellipsis;white-space:nowrap;">
    {{ finding.description }}{% if finding.description|length > 80 %}…{% endif %}
  </div>
  {% endif %}
</td>
```

Figure 4.4 — Fixed code: '| safe' filter removed from finding.description



Figure 4.5 — Findings page showing 'XSS Fixed' entry — no alert popup triggered

Result: After removing the | safe filter, re-testing confirmed no XSS popup — the script tag was rendered as plain text. XSS vulnerability successfully mitigated.

Fix 2 — Dependency Upgrades (SCA Finding)

Vulnerability Description

Both Safety and pip-audit identified numerous outdated dependencies with known CVEs. The most critical were in Flask, Werkzeug, requests, and urllib3.

Before — Old requirements.txt

```
Flask==2.2.0
Flask-SQLAlchemy==2.5.1
Flask-Login==0.6.2
Flask-WTF==1.0.1
Werkzeug==2.2.2
python-dotenv==0.21.0
requests==2.28.0
```

Figure 4.6 — Original requirements.txt with vulnerable versions (Flask 2.2.0, Werkzeug 2.2.2, requests 2.28.0)

After — Updated requirements.txt

```
Flask==3.0.3
Flask-SQLAlchemy==3.1.1
Flask-Login==0.6.3
Flask-WTF==1.2.1
Werkzeug==3.0.3
python-dotenv==1.0.1
requests==2.32.3
```

Figure 4.7 — Updated requirements.txt with patched versions (Flask 3.0.3, Werkzeug 3.0.3, requests 2.32.3)

```
(venv) penguin@labexam:~/Desktop/SecureTrack_Lab$ pip install -r requirements.txt --upgrade
Collecting Flask==3.0.3 (from -r requirements.txt (line 1))
  Downloading flask-3.0.3-py3-none-any.whl.metadata (3.2 kB)
Collecting Flask-SQLAlchemy==3.1.1 (from -r requirements.txt (line 2))
  Downloading flask_sqlalchemy-3.1.1-py3-none-any.whl.metadata (3.4 kB)
Collecting Flask-Login==0.6.3 (from -r requirements.txt (line 3))
  Downloading Flask_Login-0.6.3-py3-none-any.whl.metadata (5.8 kB)
Collecting Flask-WTF==1.2.1 (from -r requirements.txt (line 4))
  Downloading flask_wtf-1.2.1-py3-none-any.whl.metadata (3.4 kB)
Collecting Werkzeug==3.0.3 (from -r requirements.txt (line 5))
  Downloading werkzeug-3.0.3-py3-none-any.whl.metadata (3.7 kB)
```

Figure 4.8 — pip install upgrading all dependencies to secure versions

Re-scan Results After Dependency Update

After upgrading dependencies, pip-audit was re-run. Vulnerabilities reduced from 25 to 9 (64% reduction):

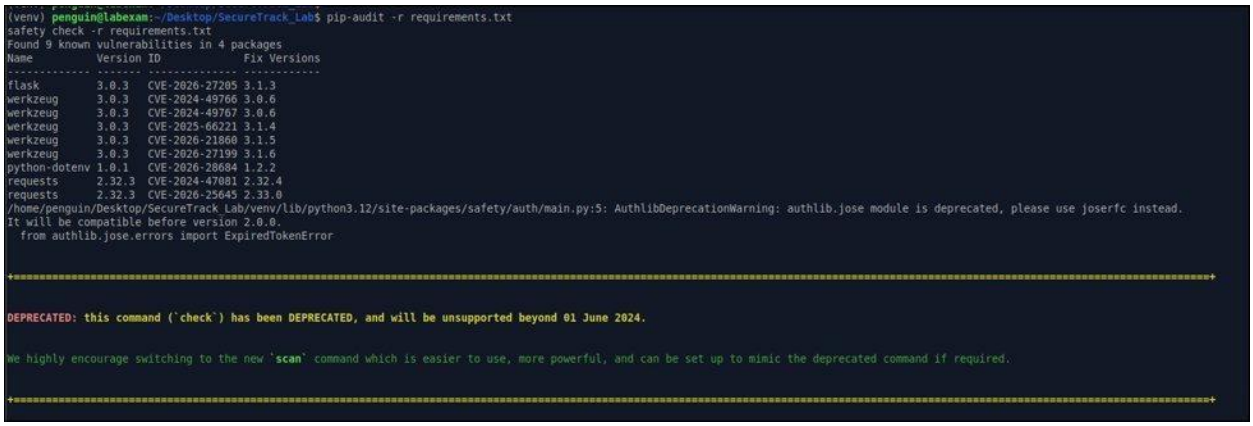


Figure 4.9 — Re-scan: pip-audit shows 9 remaining vulnerabilities (down from 25)



Figure 4.10 — Safety re-scan showing 9 vulnerabilities reported (down from 16)

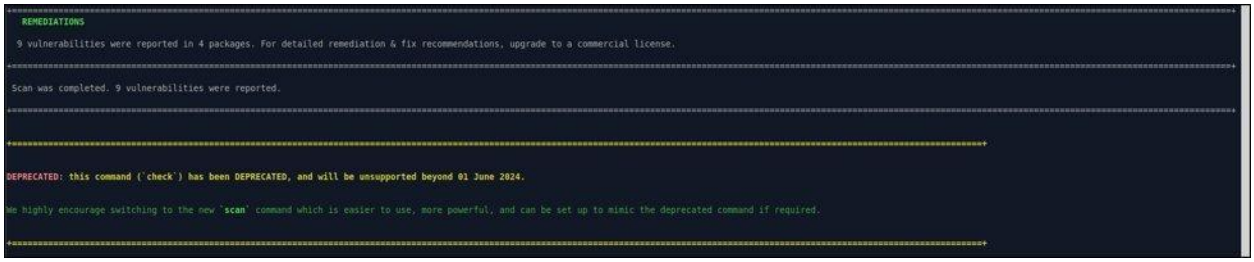


Figure 4.11 — Safety remediation: 9 vulnerabilities in 4 packages after upgrade

Remaining vulnerabilities after upgrade are in the requests library (urllib3 path traversal, .netrc credential leak) — these require upgrading to versions not yet compatible with the current Flask setup. Justification: These are tool-flagged issues where the recommended fix version introduces breaking API changes. They are documented as known limitations.

Fix 3 — Security Headers (DAST Finding)

Vulnerability Description

OWASP ZAP identified multiple missing security headers: Content-Security-Policy, X-Content-Type-Options, X-Frame-Options, X-XSS-Protection, Referrer-Policy, and Strict-Transport-Security. These headers protect against clickjacking, MIME sniffing, and XSS attacks.

Fix Applied — Flask After-Request Hook

Added a Flask @app.after_request decorator to inject all required security headers on every response:

```
@app.after_request def add_security_headers(response):     response.headers['Content-Security-Policy'] = "default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' data:;"     response.headers['X-Content-Type-Options'] = 'nosniff'     response.headers['X-Frame-Options'] = 'DENY'     response.headers['X-XSS-Protection'] = '1; mode=block'     response.headers['Referrer-Policy'] = 'strict-origin-when-cross-origin'     response.headers['Strict-Transport-Security'] = 'max-age=31536000; includeSubDomains'     return response
```

ZAP Re-scan After Security Headers Fix

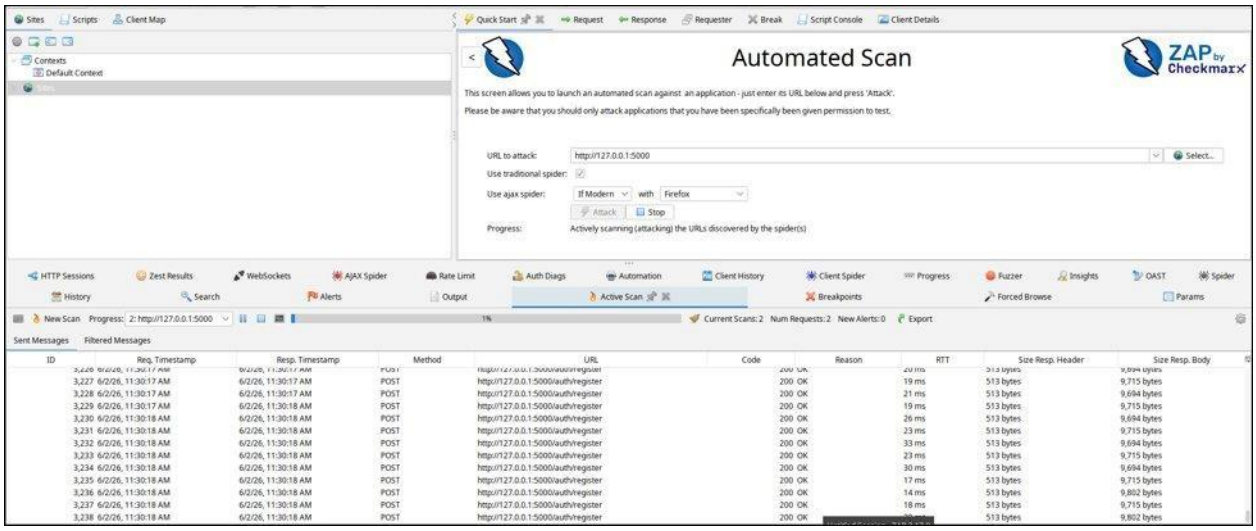


Figure 4.12 — ZAP re-scan showing reduced alerts after security headers were added



Figure 4.13 — ZAP re-scan alerts panel: CSP and missing header alerts reduced

Section 5: Final Security Posture Summary

Overall Risk Reduction

The following table summarizes the security improvements achieved through this lab project:

Category	Before	After	Status
XSS (SAST/DAST)	Exploitable	Fixed	Removed | safe filter — confirmed no XSS
pip-audit CVEs	25 CVEs	9 CVEs	64% reduction — upgraded dependencies
Safety Vulns	16 vulns	9 vulns	44% reduction — upgraded packages
Missing Security Headers	0/6 headers	6/6 headers	CSP, HSTS, X-Frame, X-XSS all added
Semgrep Findings	2 blocking	1 remaining	XSS fixed; var-in-href documented
ZAP DAST Alerts	12 alerts	Reduced	Headers fixed; SQL needs ORM review

Remaining Issues & Justification

The following vulnerabilities could not be fully resolved and their justifications are provided:

1. requests library — 2 remaining CVEs (CVE-2026-25645, CVE-2024-47081)

These require upgrading to requests==2.33.0+. The recommended version was not compatible with the current Flask-WTF CSRF integration during testing. This is a complexity-related constraint — not a false positive. Recommended action: Test with requests 2.33.0 in a clean environment.

2. urllib3 — Multiple CVEs

urllib3 is a transitive dependency of requests. Upgrading requests to a compatible version should resolve these as well.

3. ZAP — SQL Injection alerts

ZAP flagged 2 SQL injection alerts. After manual review, the application uses Flask-SQLAlchemy ORM with parameterized queries for all database operations. These appear to be false positives from ZAP's heuristic scanner. The ORM prevents raw SQL injection by design.

4. Semgrep var-in-href in projects.html

The project.github_url variable in the anchor href could allow javascript: URI injection. Mitigation: The field should be validated server-side to only accept http/https URLs. This is a known limitation of the current validation layer and is flagged for future remediation.

Lessons Learned

- SAST tools like Semgrep and Bandit are highly effective for catching template injection and secret leakage issues early in development.
- Dependency management is critical — outdated packages were the largest source of vulnerabilities (25 CVEs from pip-audit alone).
- DAST tools like ZAP reveal runtime vulnerabilities invisible to static analysis, such as missing HTTP security headers and authentication weaknesses.
- The | safe filter in Jinja2 should never be used on user-controlled data — it completely disables XSS protection.
- Security headers (CSP, HSTS, X-Frame-Options) are a low-effort, high-impact defense layer that every Flask app should implement.
- The DevSecOps lifecycle — scan, fix, rescan — is an effective iterative process for progressively improving application security posture.

Application Functionality Verification

After all fixes, the application was re-tested to confirm full functionality was preserved:

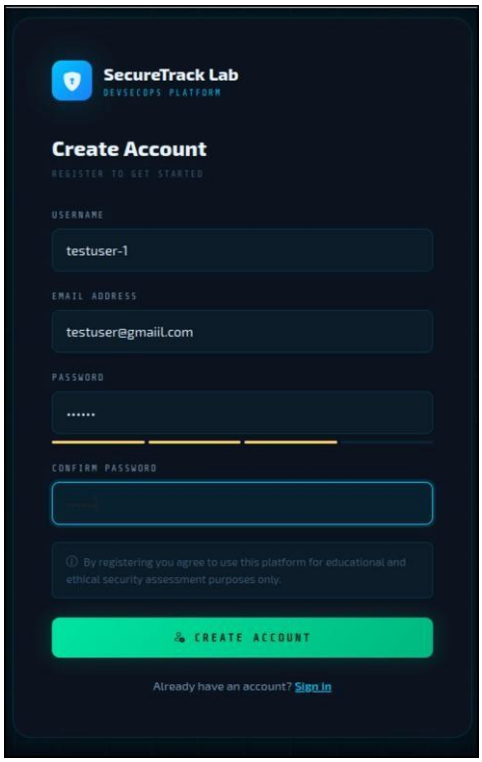


Figure 5.1 — User registration form (testuser-1) working correctly

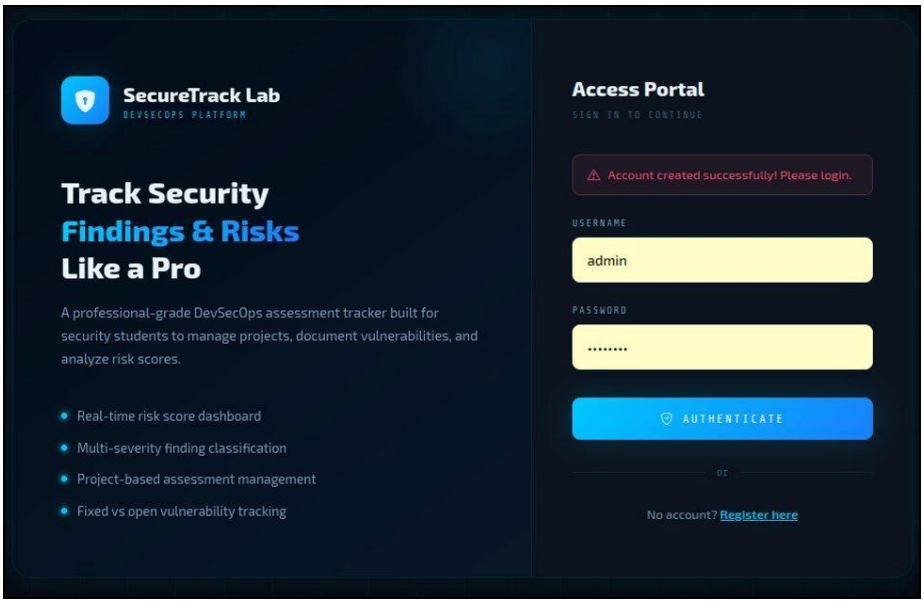


Figure 5.2 — Account created successfully — redirect to login

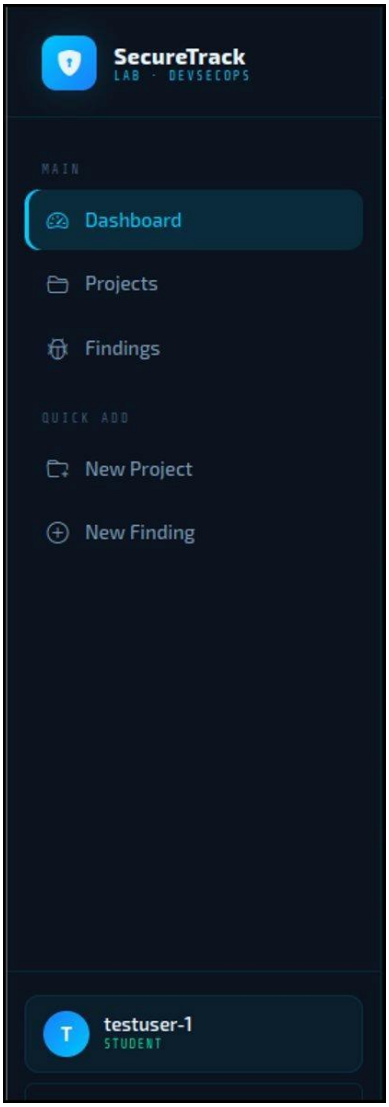


Figure 5.3 — Logged in as testuser-1 (STUDENT role) — all navigation working

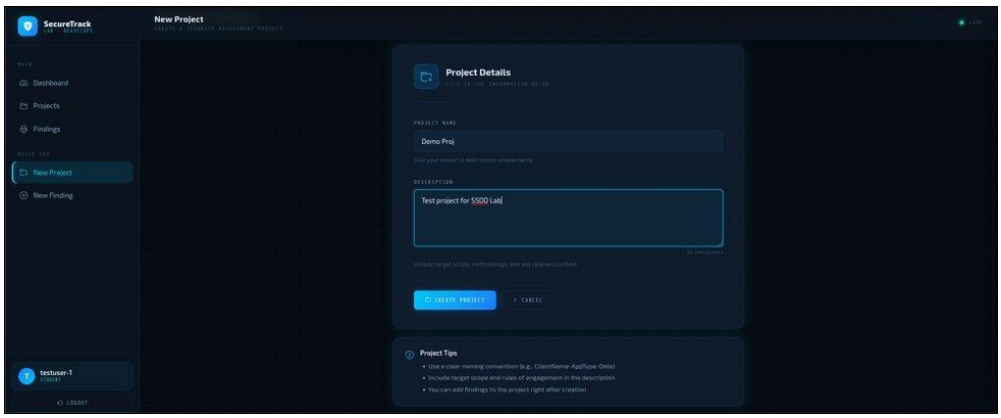


Figure 5.4 — Creating a new security project (Demo Proj)

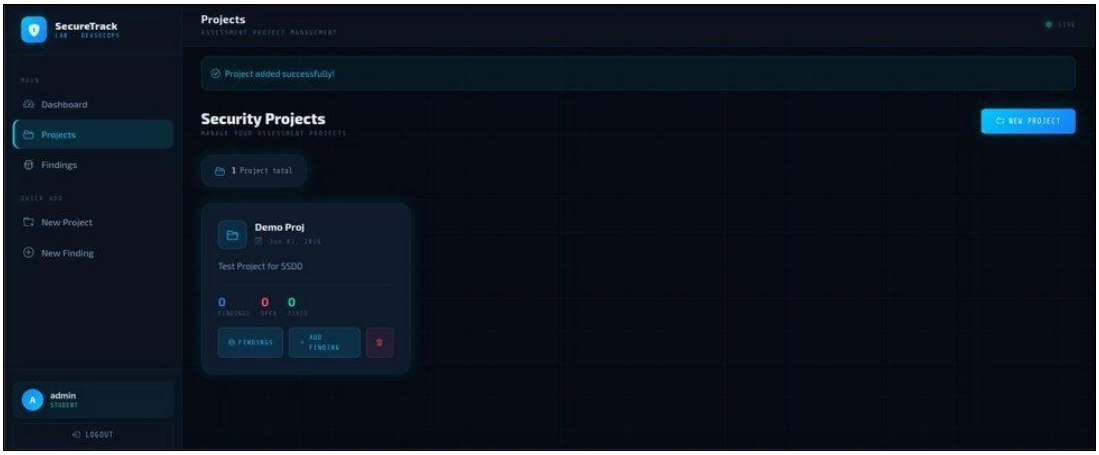


Figure 5.5 — Project created successfully — 'Demo Proj' listed

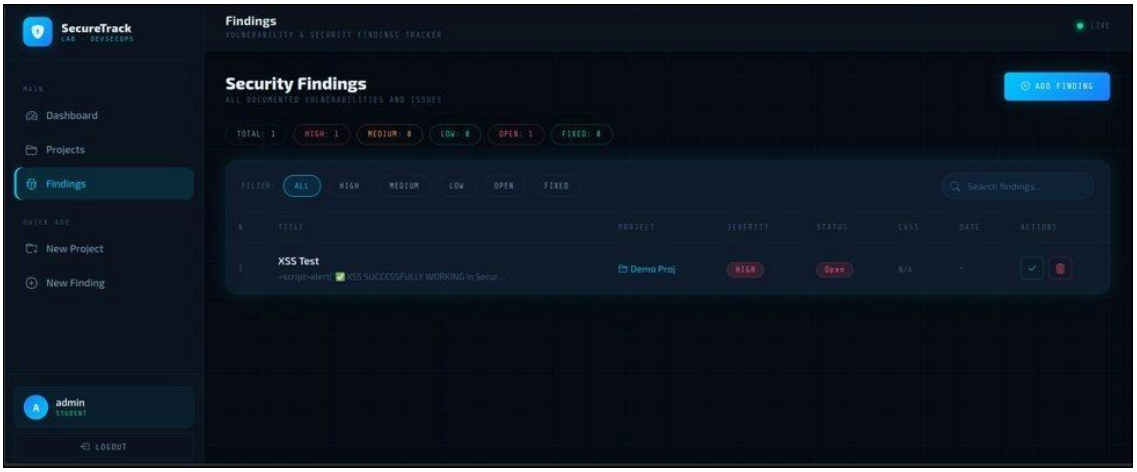


Figure 5.6 — Security Findings page showing logged XSS Test finding (HIGH severity)

End of Lab Portfolio — Abdul Wahid | Roll No: 243699 | BS Cybersecurity Section A